

Zero to Hero Masterclass:

Conquering Milestones 1 & 2

Today, we achieved two massive milestones back-to-back: **Milestone 1 (Isaac Sim Physics & USD/URDF Architecture)** and **Milestone 2 (MoveIt 2 Kinematics & ros2_control)**. This document is a complete, deep-dive record of every file changed, the physics concepts behind the errors, and exactly how we engineered the solutions to bridge the virtual and physical domains.

MILESTONE 1: Isaac Sim Physics & USD Optimization

Before we even touched MoveIt, we had to fix the raw URDF and USD files. A visually perfect robot means nothing if the physics engine cannot mathematically calculate its mass.

Issue 1: The "Exploding Robot" (Mass Ratios & Virtual Inertia)

The Problem: Upon importing the raw URDF into Isaac Sim and pressing "Play", the robot instantly exploded, vibrated violently, or disappeared from the screen (PhysX returning NaN velocities).

The Concept: PhysX 5 solves momentum transfer equations every frame. The AR5 arm links are heavy (kilograms), while the LinkerHand O6 fingers are tiny (grams). This creates a mass ratio exceeding 1:1000. When calculating the forces required to keep these links attached, the math approaches infinity, and the solver diverges.

Analogy: Trying to tie a heavy tungsten bowling ball to a piece of light cardboard with a hinge, and then swinging the bowling ball. The cardboard rips instantly.

The Solution: We injected `physxJoint:armature = 0.05` into all joints in the USD file. Armature is virtual inertia. It acts like a heavy, invisible flywheel on the motor shaft. It doesn't make the robot weigh more against gravity, but makes the math sluggish and stable.

Issue 2: The Collapsing Robot (Position Drive)

Even with stable joints, gravity instantly pulled the heavy arm down, collapsing it into a pile.

Analogy: Muscle tension. If a human passes out, their muscles lose tension and they collapse under gravity.

The Solution: We added `drive:angular:physics:stiffness = 400.0` and `damping = 40.0` to the USD joints. This acts as a virtual PD (Proportional-Derivative) controller, giving the joints the rigidity to hold their shape against gravity.

Issue 3: The Disconnected Skeleton (Articulation Root)

The Problem: The Articulation Controller in Isaac Sim refused to control the robot, or joints moved independently like loose objects.

The Concept: PhysX requires a strict hierarchical tree to solve joint constraints simultaneously.

The Solution: We applied the `PhysicsArticulationRoot` API strictly to the top-level parent (`ar5_o6_left_combined`). We also disabled gravity on the base link so the arm wouldn't plummet through the floor.

MILESTONE 2: MoveIt 2 Kinematics & The Brain

With the physical world (Isaac Sim) stabilized, we shifted to the Brain (MoveIt 2) and the Spinal Cord (ros2_control).

Issue 4: The Strict C++ Parameter Crash

The Problem: `ros2 launch ahmed_moveit demo.launch.py` crashed instantly with: `expected double, got integer`.

The Concept: The Setup Assistant generated `joint_limits.yaml` with whole numbers (e.g., `max_velocity: 1`). ROS 2 parameters are strictly typed in C++. It expected a floating-point decimal and panicked.

The Solution: We manually edited `joint_limits.yaml` and appended `.0` to every whole number (changing `1` to `1.0`).

Issue 5: The Piano Player (GripperCommand)

The Problem: MoveIt couldn't load the `hand_controller` action server.

The Concept: The Setup Assistant mistakenly classified our 6-DoF dexterous hand as a `GripperCommand`, which is strictly designed for simple 1-DoF parallel-jaw claws (open or close).

Analogy: Trying to control a concert pianist's hand using a single light switch.

The Solution: We rewrote `moveit_controllers.yaml` to classify the hand as a full `FollowJointTrajectory` controller, granting MoveIt independent mathematical control over all 6 active knuckles.

Issue 6: The Dexterous Hand Self-Collision Abort

The Problem: MoveIt instantly aborted planning for the hand, throwing `Start state in collision`.

The Concept: Dexterous hands have extremely dense collision meshes. When closing the fingers into a fist (like the ``thumbs`` pose), the bounding boxes mathematically intersect. MoveIt's default safety system detects this as a crash.

The Solution: We generated an exhaustive collision-exclusion matrix, injecting `<disable_collisions>` tags for all 13 hand links into the SRDF, telling MoveIt to legally allow the fingers to touch each other.

The Grand Integration: Dexterous Mimic Joints

Issue 7: The Mimic Joint Planning Crash

The Problem: Attempting to plan a path for the hand instantly aborted.

The Concept: The LinkerHand O6 has powered motors at the knuckles (active) and unpowered tip joints linked via tendons (passive mimic joints). A passive joint (`lh_thumb_ip`) was accidentally included in the active hand planning group. MoveIt cannot mathematically plan independent paths for passive joints slaved to other joints.

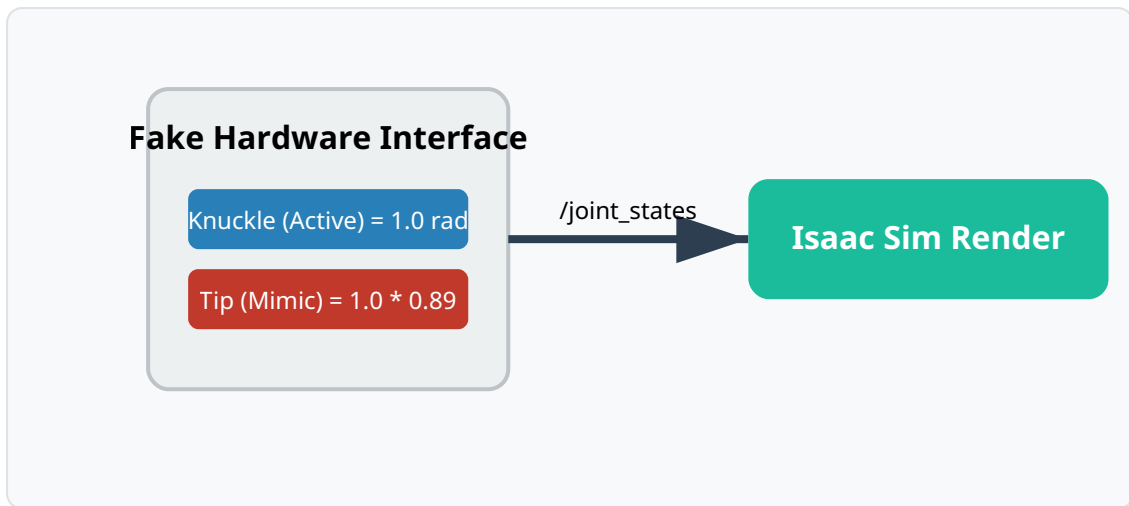
The Solution: We permanently deleted `lh_thumb_ip` from the SRDF planning group.

Issue 8: The Stiff Fingers Visual Bug

The Problem: In RViz/Isaac Sim, the knuckles bent, but the tips remained perfectly straight like stiff wooden boards.

The Concept: We are using `mock_components/GenericSystem` (fake hardware). By default, it only calculates angles for active motors. The unpowered tip angles were never broadcast to `/joint_states`, so Isaac Sim assumed they were `0.0`.

The Solution: We injected mathematical mimic tags directly into the `ros2_control.xacro`. This forced the fake hardware to calculate the unpowered tip angles in real-time.



Issue 9: The Action Graph Race Condition

The Problem: Movelt was working perfectly, but the physical robot in Isaac Sim refused to move.

The Concept: In Isaac Sim Action Graphs, white execution wires determine the strict flow of time. We originally had parallel execution wires triggering the Subscribe node and the Controller simultaneously. The Controller was firing before the Subscribe node finished downloading the new ROS 2 data.

The Solution: We rewired the graph sequentially: Tick → Subscribe (Exec Out) → Controller (Exec In).

Milestones 1 & 2 are 100% Complete. We are cleared for Milestone 3 (RL Training)!